Betriebssysteme 1 | Bsys1

Zusammenfassung

INHALTSVERZEICHNIS

1. Prozessor ... 3
   - 1.1. Instruktionen ... 3
     - 1.1.1. Codierung ... 3
     - 1.1.2. Sequenzen ... 3
     - 1.1.3. Takt / Zyklus ... 4
2. Maschinencode ... 4
   - 2.1. Intel Terminologie ... 4
   - 2.2. Formate ... 4
3. Assembler ... 5
   - 3.1. Spezifikation von Daten ... 5
     - 3.1.1. Spezifikation von Zahlen ... 5
   - 3.2. Bits ... 6
   - 3.3. Offset ... 6
   - 3.4. Labels ... 6
   - 3.5. Flat-Form Binaries ... 7
   - 3.6. Objekt-Dateien ... 7
     - 3.6.1. Symboltabelle ... 7
   - 3.7. Register ... 7
     - 3.7.1. Allzweckregister ... 8
     - 3.7.2. Spezialregister ... 8
     - 3.7.3. Übersicht ... 8
   - 3.8. Instruktionen ... 9
     - 3.8.1. Datentransfer-Operationen ... 9
     - 3.8.2. Adressierung ... 10
   - 3.9. Linker ... 11
     - 3.9.1. Einsprungspunkt ... 11
     - 3.9.2. Syscalls ... 11
     - 3.9.3. Ende des Programms ... 11
4. C ... 11
   - 4.1. Toolchain ... 11
   - 4.2. Präprozessor ... 12
     - 4.2.1. Tokenization ... 12
     - 4.2.2. Präprozessor-Direktiven ... 12
     - 4.2.3. Includes ... 13
     - 4.2.4. Macros ... 13
   - 4.3. Compiler ... 13
     - 4.3.1. clang commands ... 13
     - 4.3.2. Deklaration und definition ... 14
     - 4.3.3. Variablen ... 14
     - 4.3.4. Typen ... 15
     - 4.3.5. Ausdrücke ... 16

5. Arithmetische und logische Operationen ... 16
5.1. Logische Operationen ... 16
5.1.1. NAND ... 16
5.1.2. Assembler ... 17
5.1.3. C ... 17
5.2. Arithmetische Operationen ... 17
5.2.1. Assembler ... 17
5.2.2. C ... 18
5.2.3. Links- und Rechts-Shift ... 18
5.2.4. Multiplikation ... 19
5.2.5. Division ... 19
5.2.6. C ... 19
5.2.7. Optimierungen ... 19
6. Kontrollfluss ... 20
6.1. Flags ... 20
6.2. Condition Codes (CC) ... 20
6.2.1. Unsigned ... 21
6.2.2. Signed ... 21
6.2.3. Intel CCs ... 21
6.3. jmp ... 22
6.4. Vergleiche ... 22
6.4.1. cmp ... 22
6.4.2. test ... 22
6.5. Bedingte Anweisungen ... 22
6.5.1. Bedingte Sprünge ... 22
6.5.2. Schleifen ... 23
6.6. Pointer-addition ... 23
6.7. Funktionen ... 23
6.7.1. Parameter ... 23
6.7.2. Stack ... 24
6.7.3. CALL/RET ... 25
6.7.4. Funktion ... 26
6.7.5. Argumente ... 26
6.7.6. Frame Pointer ... 26
6.7.7. Calling Convention ... 26
6.7.8. Zwischenergebnisse ... 27
6.7.9. C ... 27

WARN:
This document is still under development and will be completed around 1.2.2027

1. PROZESSOR

Der Prozessor und Speicher sind getrennt und sind durch einen Speicherbus verbunden. Dieser umfasst:

Adressbus: enthält Adresse der Speicherzelle, auf die zugegriffen werden soll

Datenbus: enthält Daten, die aus Speicherzelle gelesen wurde bzw. in diese geschrieben werden soll

Steuersignale: die z.B. angeben, ob gelesen oder geschrieben werden soll

Schreibvorgang	*Lesevorgang*
1) Prozessor legt Adresse auf Adressbus	1) Prozessor legt Adresse auf Adressbus
2) Prozessor legt Daten auf Datenbus	2) Prozessor aktiviert Speicherbus zum Lesen
3) Prozessor aktiviert Speicherbus zum Schreiben	3) Speicher legt Daten auf Datenbus

- Jeder Prozessor enthält eine kleine Menge an Speicher: die ***Register***
- Jeder Prozessor besteht aus vielen kleinen Bausteinen, die jeweils eine ***Operation*** durchführen können
  - Operationen können Register und/oder den Speicherbus lesen und/oder schreiben
  - Operationen können 0 bis n Operanden (Register, Adressen, Daten) haben
  - Die Anzahl und Art von Operationen ist immer prozessor-spezifisch, aber auf den meisten Prozessoren ähnlich

1.1. INSTRUKTIONEN

Eine ***Instruktion*** ist die Kombination einer Operation mit Operanden, z.B. «Kopiere Wert aus Register x in Register y». `mov y, x`

1.1.1. Codierung

Operationen und Register werden durchnummeriert. Gemeinsam mit Daten- und/oder Adressoperanden ergibt sich eine Bytesequenz.

Beispiel 1: **«Addiere 12345678_h zu Register 1» auf Intel 64 Architektur**

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5
81_h	$C1_h$	78_h	56_h	34_h	12_h
Opcode		12345678_h			

Die Gesamtheit aller möglichen (binären) Instruktionscodes eines Prozessors (Maschine) ist sein Maschinencode und wird vom Hersteller festgelegt.

1.1.2. Sequenzen

Die Sequenzen der Instruktionen können an mehreren Orten liegen:
- Hart-codiert im Prozessor: Sequenz kann nach Produktion des Prozessors nie geändert werden
- Im Hauptspeicher: Sequenz kann beliebig geändert werden, muss über den Speicherbus abgefragt werden

Der Prozessor hat ein spezielles Register, das die Adresse des nächsten Befehls im Hauptspeicher enthält (Befehlszeiger / ***instruction pointer*** IP / ***program counter*** PC).

1.1.3. Takt / Zyklus

Der gesamte Computer ist getaktet: Alle Bausteine (auch im Prozessor, im Speicher etc) erhalten ein Takt-Signal (***Clock***)

Über mehrere Takte hinweg führt der Prozessor folgende Schritte aus (***Zyklus***):
1) Prozessor fordert Instruktion ab der Adresse an, die im Befehlszeiger steht
2) Prozessor decodiert Operation und Operanden aus Instruktion
3) Prozessor wählt mit Operation korrespondierenden Baustein aus
4) Aktiver Baustein liest ggf. aus den Registern
5) Aktiver Baustein führt ggf. Berechnung aus
6) Aktiver Baustein schreibt ggf. in die Register
7) Prozessor erhöht Befehlszeiger entsprechend der Länge der Instruktion

Diese Schritte können stark parallelisiert werden (***Pipelining***)

2. MASCHINENCODE

Intel 64 Prozessoren basieren auf einer Little-Endian-Architektur. Es gibt 2 Modi: 32 Bit (Legacy) und 64 Bit, befassen uns hier aber nur mit 64 Bit.

2.1. INTEL TERMINOLOGIE

Begriff	***Definition***
Byte	8 Bit
Word	2 Byte / 16 Bit
Doubleword	4 Byte / 32 Bit, auch DWord
Quadword	8 Byte / 64 Bit, auch QWord
Double Quadword	16 Byte / 128 Bit, auch DQWord

Intel-Prozessoren verwenden als kleinste Einheiten einzelne Bytes (8 Bit). Ein einzelnes Bit muss immer mit dem gesamten Byte verarbeitet werden. Dargestellt werden die Bytes als 2 hexadezimale Stellen s_0 und s_1, wobei die höherwertige Ziffer immer links steht ($s_1 s_0$).

Ein Word hat 4 hexadezimale Stellen s_0 bis s_3, wobei es üblich ist, die Stellen **von rechts nach links** ($s_3 s_2 s_1 s_0$) zu schreiben.

2.2. FORMATE

Die Bytes werden im ***Big-Endian*** Format **von rechts nach links** und im ***Little-Endian*** Format **von links nach rechts** geschrieben.

Big-Endian

1b	0b
CA_h	FE_h
i	$i+1$

Little-Endian

0b	1b
FE_h	CA_h
i	$i+1$

Big-Endian	***Little-Endian***
Zahlen beginnen mit dem «grossen Ende», dem ***MSByte*** (Most Significant)	Zahlen beginnen mit dem «kleinen Ende», dem ***LSByte*** (Least Significant)

3b	2b	1b	0b
87_h	65_h	43_h	21_h
i	$i+1$	$i+2$	$i+3$

0b	1b	2b	3b
21_h	43_h	65_h	87_h
i	$i+1$	$i+2$	$i+3$

Vorteil von Little-Endian ist, dass man ein DWord mit Nullen in den höherwertigen Stellen als Word oder Byte mit derselben Adresse verwenden kann.

Big-Endian

3b	2b	1b	0b
00_h	00_h	00_h	56_h
i	$i+1$	$i+2$	$i+3$
			Byte
		Word	
DWord			

Little-Endian

0b	1b	2b	3b
56_h	00_h	00_h	00_h
i	$i+1$	$i+2$	$i+3$
Byte			
Word			
DWord			

3. ASSEMBLER

Der Assembler ist ein Programm, das textuelle Befehle in Maschinencodes übersetzt. Konventionen hängen massgeblich vom Hersteller des Assemblers ab, selbst für denselben Prozessor. Wir verwenden in diesem Modul den Netwide Assembler NASM .

3.1. SPEZIFIKATION VON DATEN

Quellcode	***Resultat im Binär-Output***
`db 48`	das Byte `48d`
`db 0x35, 0h21, 049h`	die drei Byte `35h, 21h, 49h`
`db 'a'`	das Byte mit dem ASCII-Code von `a` = `61h` ≡ `db 0x61`
`db 'Hallo'`	die ASCII-Codes von `H`, `a`, `l`, `l` und `o` ≡ `db 0x48, 0x61, 0x6c, 0x6c, 0x6f`

Statt einzelnen Bytes können auch Words, DWords und QWords spezifiziert werden, Endianness muss dann zwingend beachtet werden:

```
dw 0x2135 ; = db 0x35, 0x21
dd 0x2135 ; = db 0x35, 0x21, 0x00, 0x00
dq 0x2135 ; = db 0x35, 0x21, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
```

3.1.1. Spezifikation von Zahlen

$ Präfix auch möglich.

Base	*Suffix*	*Präfix*
Hex	H / X	0h / 0x
Dec	D / T	0d / 0t
Oct	Q / O	0q / 0o
Bin	B / Y	0b / 0y

3.2. BITS

NASM übersetzt fast jede Anweisung direkt in Binärzahlen und schreibt diese als Sequenz von Bytes in die Zieldatei

```
nop           ; 90h
not rax       ; 48h F7h D0h
mov rax, rbx  ; 48h 89h D8h
```

Traditionell arbeitet der NASM im historischen 16-Bit-Modus, Operationen haben hier andere Opcodes als im 64-Bit-Modus. Der 64-Bit-Modus wird mit der Direktive `BITS 64` aktiviert (möglichst als 1. Zeile)

3.3. OFFSET

Nummeriert man die Bytes in der Ausgabe-Sequenz fortlaufend, erhält jedes Byte einen ***Offset*** (Adresse, Index).

```
...
dd 0x12345678
...
```

78_h	56_h	34_h	12_h
i	$i+1$	$i+2$	$i+3$

$ bezeichnet den aktuellen Offset, bevor die von der aktuellen Zeile generierten Bytes in die Ausgabe-Sequenz geschrieben werden.

```
db 'BSys 1?!'
dq $
; erzeugt                  42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00
```

3.4. LABELS

Am Anfang jeder Zeile in der Quelle kann ein Label stehen, das **nicht** in Maschinencode übersetzt wird:

```
my_text: db 'BSys 1?!'
current: dq current
; erzeugt auch             42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00
```

Intern assoziiert der Assembler das Label w mit dem Offset $A(w)$ des nachfolgenden Befehls in der Ausgabedatei und ersetzt jede Verwendung von w in der Quelle durch $A(w)$ in der Ausgabe-Sequenz.

42_h	53_h	79_h	73_h	20_h	31_h	$3f_h$	21_h	08_h	00_h	00_h	00_h	00_h	00_h	00_h	00_h
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
^ my_text								^ current							

NASM kann während der Übersetzung einfache Arithmetik ausführen:

```
length: dq after_my_text - my_text
my_text: db 'BSys 1?!'
after_my_text:
; erzeugt                 08 00 00 00 00 00 00 00 42 53 79 73 20 31 3f 21
```

3.5. FLAT-FORM BINARIES

Per default erzeugt der NASM sogenannte ***Flat-Form Binaries*** (plain/pure/raw): Die reine Bytesequenz analog zum Quell-Text ohne Zusatzinformationen.

3.6. OBJEKT-DATEIEN

Objekt-Dateien enthalten darüber hinaus noch weitere Informationen, die später benötigt werden, u.a. die Symboltabelle.

Exportierte Symbole: mit `global` deklarierte Label

Importierte Symbole: mit `extern` deklarierte Label, haben keinen definierten Wert innerhalb der Objekt-Datei

Auf 64-Bit Linux sind Objekt-Dateien im Format ELF64 mit Endung `*.o`

```
nasm -f elf64 prog.asm -o prog.o
```

ELF64 kann mit dem Tool `objdump` analysiert werden:

```
objdump -t -d -Mintel prog.o
```

3.6.1. Symboltabelle

Die Symboltabelle enthält einen Eintrag pro Symbol.

```
global x, y
extern z
dq 0, 0
w: dq 0x12345678
x: dq 0xCAFEFACE
y: dq z
```

Offset	Attribute (g)lobal, (l)ocal	Sektion	Zusatzinformationen	Name
0000000000000010	l	.text	0000000000000000	w
0000000000000000		*UND*	0000000000000000	z
0000000000000018	g	.text	0000000000000000	x
0000000000000020	g	.text	0000000000000000	y

Das als extern deklarierte Label z erscheint als `*UND*` (undefined)

3.7. REGISTER

Es wird zwischen «General Purpose» und «Special Purpose» Register unterschieden.

General Purpose: Kann nach belieben verwendet werden

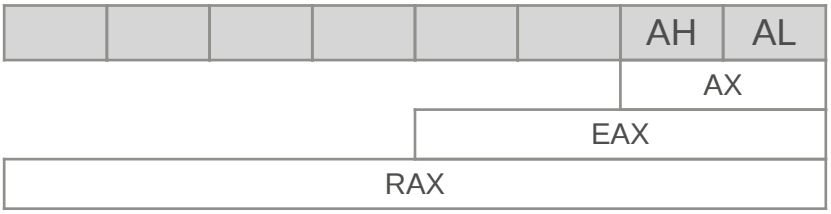
Special Purpose: Wird von der CPU für einen spezifischen Zweck verwendet, nur gewisse Operationen können darauf zugreifen.

Ursprünglich hatten Intel-Prozessoren (ab dem 8088) 16-Bit-Register, die in zwei 8-Bit-Register (`AH` `AL`) unterteilt wurden.

Man kann sowohl `AX` als auch `AL` oder `AH` in Instruktionen verwenden.

Mit der 32-Bit-Architektur wurden diese Register um namenlose 16-Bit erweitert (→ `EAX`), Mit der 64-Bit-Architektur um weitere 32-Bit (→ `RAX`).

Mit der ISA Erweiterung enstand aber auch eine neue Limitierung: `ah`, `bh`, etc. können nicht in allen Situation verwendet werden.



3.7.1. Allzweckregister

General Purpose Registers (GPRs)

RAX: Accumulator, für einige Rechenoperationen das einzige mögliche Register

RCX: Counter für Schleifen und Stringoperationen

RDX: Pointer für I/O-Operationen

RBX: Datenpointer

RSI, RDI: Quell- und Zielindizes für Stringoperationen

RSP: Stackpointer, Adresse des allozierten Stacks

RBP: Basepointer, Basis des Stackframe der Funktion

R8 - R15: Zusätzliche Register

Wichtig: Die Verwendungszwecke sind reine Konvention und nicht zwingend.

3.7.2. Spezialregister

Special Purpose Registers (SPRs)

RIP: Instruction Pointer, zeigt auf die nächste Instruktion

SS, CS, DS, ES, FS, GS: Segment Register (kaum noch verwendet)

RFLAGS: Flags die spezifische Situationen markieren (z.B. Integer Overflow)

CR0 - CR15: Control-Registers um die CPU zu konfigurieren

GDTR: Enthält die Adresse der GDT - verwendent für VM

LDTR: Enthält die Adresse der LDT - verwendent für VM

IDTR: Enthält die Adresse der IDT - verwendent für VM

TR: Enthält die Adresse des TSS - verwendet für Task Switching

3.7.3. Übersicht



RAX	EAX	AH AL / AX
RCX	ECX	CH CL / CX
RDX	EDX	DH DL / DX
RBX	EBX	BH BL / BX
RSP	ESP	- SPL / SP
RBP	EBP	- BPL / BP
RSI	ESI	- SIL / SI
RDI	EDI	- DIL / DI
R8	R8D	- 8B / R8W
R...	R...D	- ...B / R...W
R15	R15D	- 15B / R15W

Kompatibilität: 64-Bit ISA Instruktionen haben REX Präfix (Der NASM fügt diesen ein wenn nötig). Betroffen sind `SPL, BPL, SIL` und `DIL`.

Bsp: `mov ah, sil` entspricht `REX mov ah, sil`

Würde `mov spl, sil` bedeuten, wird nicht vom Assembler akzeptiert. Die Instruktionen könnten kodiert werden, hätten aber falsche Semantik.

3.8. INSTRUKTIONEN

Operationen benötigen unterschiedlich lange: Die schnellsten benötigen 1 Prozessorzyklus, die langsamsten mehrere 100.

Operationen, die auf den Speicher zugreifen müssen, müssen auf den Speicher warten: Ist der Operand im Cache: 4 bis 70 Zyklen, ansonsten: mehrere 100 Zyklen

Bestimmte Operationen könnten nur mit immensem Aufwand schneller gemacht werden (z.B. Division)

Instruktionen sind Binärzahlen, die die Operation und die Operanden codieren. Sie können auf Intel 64 unterschiedlich lang sein (1 bis 15 Byte). Die Anzahl und Grösse der Parameter hängen von der Operation ab. Die Länge einer Instruktion ist nicht in der Sequenz enthalten, eine Sequenz muss von Anfang an Instruktion für Instruktion durchgegangen werden, um diese richtig decodieren zu können.

3.8.1. Datentransfer-Operationen

`mov ziel, quelle` Kopiert in das Ziel von der Quelle. Nach Ausführung der Operation enthalten Quelle und Ziel den gleichen Wert.

In ein ***Register*** kann man kopieren:

Woher	*Befehl*	*Auswirkung*
Von einem anderen Register	`mov rax, rbx`	Setze Inhalt von `rax` gleich Inhalt von `rbx`
Eine Konstante	`mov rax, 0x8000`	Setze Inhalt von `rax` gleich `8000h`
Vom Speicher	`mov rax, [0x8000]`	Setze Inhalt von `rax` gleich Inhalt von `8000h ... 8007h`

In den ***Speicher*** kann man kopieren:

Woher	*Befehl*	*Auswirkung*
Von einem Register	`mov [0x800], rbx`	Setze Inhalt von `8000h ... 8007h` gleich Inhalt von `rax`
Eine Konstante	`mov qword [0x8000], 5`	Setze Inhalt von `8000h ... 8007h` gleich `5`

aber **nicht** direkt vom Speicher in den Speicher.

Operandengrössen können explizit mit `byte, word, dword, qword`, etc angegeben werden.

Generell können die Operanden 8 Bit, 16 Bit, 32 Bit oder 64 Bit betragen, die Operanden müssen aber gleich gross sein, z.B.

`mov eax, ebx` OK, beide Register 32-Bit gross

`mov eax, rbx` Fehler, `eax` 32 Bit, `rbx` 64 Bit

Im Maschinencode gibt es für jede Operandengrösse eine eigene Instruktion

Bit	*Befehl*	*Code*
8	`mov al, bl`	`88 D8`
16	`mov ax, bx`	`66 89 D8`
32	`mov eax, ebx`	`89 D8`
64	`mov rax, rbx`	`48 89 D8`

3.8.2. Adressierung

Speicherstellen können auf verschiedene Weisen spezifiziert werden.

Displacement: `[ a ]` Die Adresse `a` der Speicherstelle folgt unmittelbar

```
mov rax, [0x8000]
```

Base (Register): `[ r ]` Die Adresse der Speicherstelle steht in einem Register `r`

```
mov rbx, 0x8000
mov rax, [rbx]
```

Scaled Index: `[ i * s ]` Die Adresse `i * s` besteht aus einem Index `i` (Register) skaliert mit einer Konstante `s` (1,2,4 oder 8)

```
mov rcx, 0x1000
mov rax, [rcx * 8]
```

Jede Summe der drei vorherigen: Alle drei Adressierungsmodi können beliebig addiert werden

```
mov rbx, 0x4000
mov rcx, 0x1000
mov rax, [0x2000 + rbx + rcx * 2]
```

Wenn man nur die Adresse berechnen möchte, kann man die Operation `lea` (Load Effective Address) verwenden (greift nicht auf Speicher zu):

```
mov rbx, 0x4000
mov rcx, 0x1000
lea rax, [0x2000 + rbx + rcx * 2]
```

3.9. LINKER

Programme werden üblicherweise aus mehreren Assemblerdateien generiert. Der Assembler erzeugt aus einer Assemblerdatei eine Objekt-Datei. Der Linker `ld` erstellt aus einer oder mehreren Objekt-Dateien ein Executable.

Mit `ld -r prog.o prog2.o -o prog3.o` können wir partiell linken: Jedes Symbol erhält einen eigenen Platz im gelinkten Objekt; globale Symbole werden an einen neuen Offset verschoben. Durch `-r` wird kein Executable erzeugt, sondern eine weitere Objekt-Datei.

3.9.1. Einsprungspunkt

In jedem Executable muss der Einsprungspunkt (entry point) definiert werden (die Adresse, auf die der IP gesetzt wird, wenn das Programm gestartet wird). Die wird vom Linker gesetzt und ist standartmässig die Adresse des Labels `_start`, Kann auf der Kommandozeile geändert werden

```
ld -e main my_prog.o -o my_prog
```

3.9.2. Syscalls

Die Instruktion `syscall` übergibt die Ausführung an das OS. Wenn das OS fertig ist, geht es an der Stelle nach dem Syscall weiter. In `rax` übergibt man den Code für die OS-Funktion, für allfällige Parameter werden auf Intel 64 verwendet: `rdi, rsi, rdx, r10, r8, r9`. Codes und Parameter können je nach Architektur ändern; sind aber im allgemeinen recht konstant

3.9.3. Ende des Programms

Programme müssen explizit beendet werden: das OS weiss nicht, wann das Programm zuende ist. Dazu gibt es den OS-Syscall `exit`, üblicherweise Code `60` mit einem einzigem Parameter: einem 8-Bit Exit-Code. Ein Programm hat also immer folgenden Rahmen:

```
global _start
_start:
...
mov rax, 60     ; syscall exit
mov rdi, 0      ; exit code
syscall
```

4. C

4.1. TOOLCHAIN

Die Bestandteile der C Toolchain sind:
1) C Präprozessor
2) C Compiler
3) Assembler
4) Linker



Es lassen sich 3 Sprachebenen unterscheiden:

Präprozessor: definiert Direktiven, die im Programm vor dem eigentlichen Übersetzen als Textersetzung durchgeführt werden

Basiskonstrukte: bestimmen das Grundgerüst eines Programms, z.B. Variablen, Schleifen, Verzweigungen.

Standardbibliotheken: stellen Funktionen und Typen bereit, die die Basis-Funktionalität enthalten

4.2. PRÄPROZESSOR

Der Präprozessor verarbeitet die Input-Datei in mehreren Durchläufen. In jedem Durchlauf verarbeitet er die gesamte Datei einmal.

1) Entfernen aller Kommentare und umwandeln fortgesetzter Zeilen, die mit \ enden, in eine einzige Zeile.
2) Tokenization
3) Preprocessor directives + Macro expansion

4.2.1. Tokenization

Es gibt 5 Klassen von Tokens:

Bezeichner (identifiers): beginnt mit einem Buchstaben (a-zA-Z) oder _ gefolgt von einer Sequenz aus Buchstaben, _ oder Ziffern (0-9)

Präprozessor-Zahlen: Beginnt mit einer Ziffer Gefolgt von einer Sequenz aus Ziffern, Buchstaben, _, ., oder Exponenten (e+, E+, e−, E−, p+, P+, p−, P−). Vor der ersten Ziffer kann auch ein Punkt . stehen.

String- und Character-Literale: String-Literale beginnen und enden mit ", Character-Literale beginnen und enden mit '

Operatoren und Satzzeichen (punctuators): Jede der folgenden Zeichen bzw. Zeichenkombinationen gilt als Punctuator:

```
( ) [ ] { } . , : ; ? ... → # ##
= + - * / % & | ^ ~ ! << >> ≔ ≠ < > ⩽ ⩾ && ||
*= ⁄= %= += -= ++ -- ⇐≔ ⇒≔ &= ⊨ ^=
```

Der Präprozessor ist greedy, d.h. er versucht immer das grösstmögliche Token zu bilden.

Sonstige:

> ***TODO:***
> ?

4.2.2. Präprozessor-Direktiven

Ist das erste Token auf einer Zeile #, wird das nächste Token als Direktive interpretiert. Beide Token werden entfernt und die entsprechende Direktive ausgeführt. Die wichtigsten Direktiven sind:

include: Fügt einen Header ein

define: Definiert ein Makro

if: Bedingte Kompilation

else: Alternativer Zweig der bedingten Kompilation

endif: Ende der bedingten Kompilation

4.2.3. Includes

Präprozessor öffnet die Datei anhand des nächsten Tokens
– `#include <file.h>` sucht nur in den Systemverzeichnissen
– `#include "file.h"` sucht erst im aktuellen Verzeichnis und dann in den Systemverzeichnissen

Präprozessor führt Durchläufe 1 bis 3 für file .h durch und setzt Arbeit nach der Direktive in Orignaldatei fort. Der Präprozessor kann dadurch mehrere Dateien zu einer ***Translation Unit*** zusammenführen

4.2.4. Macros

Es gibt objektartige und funktionsartige Makros. Objektartige Makros haben keine Parameterliste.

```
#define ANSWER 42
```

Der Präprozessor ersetzt im Programmtext nach der Definition des Makros jedes Token, das dem Makronamen entspricht, durch die Tokenliste. Nach der Ersetzung durchsucht der Präprozessor die Ersetzung auf weitere Makronamen und ersetzt diese. Taucht der eigene Makroname in der Ersetzung auf, wird er nicht ersetzt, um infinite Rekursion zu verhindern.

4.3. COMPILER

Der Compiler übersetzt eine C-Datei in eine Assembler-Datei.

Eine C Translation Unit ist eine Folge von Deklarationen und Definitionen von
– Globalen Variablen
– Funktionen
– Typen

***Beispiel** 2:*

```
static int x = 0;      // [00] 8b 04 25 00 00 00 00    mov eax, [x]
extern void g();
void f() {
  if (x == 0) {        // [07] 85 c0                   test eax, eax
                       // [09] 75 05                   jne 0x10
    g();               // [0b] e8 00 00 00 00          call g
  }                    // [10] c3                      ret
}
```

4.3.1. clang commands

Begriff	***Definition***
`clang prog.c -o prog`	Gesamte Toolchain
`clang prog.c`	Ohne Linker-Schritt, erzeugt `prog.o`
`clang -S prog.c`	Ohne Linker oder Assembler, erzeugt `prog.s`
`clang -masm=intel -S prog.c`	NASM-Kompatibles assembly
`clang -E prog.c prog.prep.c`	Nur Präprozessor

4.3.2. Deklaration und definition

Deklarationen haben keinen direkten Einfluss auf den erzeugten Byte-Stream (ähnlich wie Label in Assembler). Innerhalb einer Translation Unit darf jeder Bezeichner beliebig oft deklariert werden, solange die Deklaration gleich ist.

Eine Entität darf in einer Translation Unit nicht mehrfach definiert werden, auch nicht wenn die Definition exakt gleich ist.

4.3.3. Variablen

global	*extern*	*static*

Für jede globale Variable wird im Programm Speicher fix reserviert. Globale Variablen ohne Initialwert werden mit 0 initialisiert.
Globale Variablen werden standardmässig exportiert:
`int c = 5;`
entspricht in Assembler
`global c`
`c: dd 5`
	Globale Variablen, die aus anderen Objekt-Dateien verwendet werden sollen, werden mit extern deklariert:
	`extern int c;`
	entspricht in Assembler
	`extern c`
		Variablen, die nicht exportiert werden sollen, werden mit static bezeichnet:
		`static int c = 5;`
		entspricht in Assembler:
		`c: dd 5`

Bez. in Obj.-Datei	*Deklaration in Asm*	*Bez. in C*	*Deklaration in C*
lokal	–	global, internal Linkage	static
global	global	global, external Linkage	–
–	extern	extern	extern

4.3.3.1. Quirks

```
// a.c
extern int x;
// b.c
short x;
// Wird kompiliert und gelinkt, kann aber crashen
// Im besten Fall passiert ein segfault
// Sollte in header Files definiert und überall inkludiert werden

static int x = 4;
extern int x;
// Compiler legt Speicher an für static int x, extern int x macht nichts
```

4.3.3.2. Objekte

Objekt: ist ein zusammenhängender Speicherbereich; sein Inhalt kann als Wert interpretiert werden.

Wert: Genaue Bedeutung des Inhalts eines Objekts, wenn dieses mit einem Typ versehen ist.

Der Inhalt eines Objekts kann verschieden interpretiert werden.

4.3.4. Typen

C kennt verschiedene Arten von Typen:
– Basistypen
– Abgeleitete Typen
– Enumerationen

Typen beschreiben Objekte bzw. deren Eigenschaften:
– Jeder Typ `T` hat eine feste Objektgrösse `sizeof(T)`
  – `sizeof(T)` wird in Vielfachen von `char` angegeben, genaue Grösse in `CHAR_BIT` in `<limits.h>`. Bsp: Anzahl bits eines Typen `T` ist `sizeof(T) * CHAR_BIT`
– Bei Operationen erzwingt C Typgleichheit der Operanden
– Typinformationen existieren nur auf Sprachlevel C (Nach der Übersetzung in Assembler gibt es keine Typen mehr, sondern nur noch Speicher)

Der Compiler kann dank Typen statische checks durchführen, automatisch die richtige Grösse für bestimmte Objekte allozieren und die richtigen Operationen auswählen, z.B. unsigned oder signed Multiplikation.

Typen hängen in C von der verwendeten Maschine (und dem Compiler) ab, insbesondere ihre Grösse.

Vorteil: C-Programme laufen auf jeder Maschine möglichst effizient

Nachteil: Plattform-Unabhängigkeit von C-Programmen erfordert sehr viel Knowhow

4.3.4.1. char, signed char, unsigned char

– `char` ist wie `signed char` oder `unsigned char`, je nach Zielsystem und Implementierung, jedoch trotzdem ein eigener Typ
– mindestens 8 Bit gross (kleinste Einheit von Bits, die einzeln adressierbar sind)

8 Bit auf Intel 64, aber z.B. 32 Bit auf Analog Devices SHARC

4.3.4.2. integer

Es gibt die folgenden 5 Signed-Integer-Basistypen:

signed char: $\geq$ 8 Bit

short int: $\geq$ 16 Bit

int: $\geq$ 16 Bit und $\geq$ `short`, soll der «natürlichen» Grösse der Architektur der Ausführungsumgebung entsprechen

long int: $\geq$ 32 Bit und $\geq$ int

long long int: $\geq$ 64 Bit und $\geq$ long

Bei den letzten 4 Typen kann in einer Deklaration `signed` davor geschrieben werden und `int` kann auch weggelassen werden.

Zu jedem Signed-Integer-Basistypen gibt es das `unsigned`-Gegenstück, das die gleiche Grösse hat.

4.3.4.3. void

`void` hat keine Werte und wird verwendet, wenn keine Werte zulässig sind
– wenn eine Funktion keinen Rückgabewert hat
– wenn eine Funktion keine Argumente hat
– untypisierte Pointer (können nicht dereferenziert werden)

4.3.4.4. Abgeleitete Typen

Array: Mehrere Elemente vom gleichen Typ nacheinander

Struct: Mehrere Elemente beliebiger Typen nacheinander

Union: Mehrere Elemente beliebiger Typen im selben Speicherbereich

Pointer: Adresse eines Objekts

`void *` ist ein void-Pointer (Adresse eines Objekts, dessen Typ nicht bekannt ist)

Funktion: Adresse einer Funktion

4.3.5. Ausdrücke

Jeder Ausdruck hat einen Typen, den der Compiler aus den Operanden ableitet. Primäre Ausdrücke sind Konstanten und Literale sowie Bezeichner. Alle anderen Ausdrücke werden durch Operatoren und Casts erzeugt.

4.3.5.1. Integer-Konstanten

Dezimalzahl: Folge von Ziffern (`0..9`), die **nicht** mit `0` beginnt

Oktalzahl: Folge von Ziffern (`0..7`), die mit `0` beginnt

Hexadezimalzahl: Folge von Ziffern (`0..F`), die mit `0x` beginnt

Es kann ein Typen-Suffix angehängt werden: `l, ll, u, ul, ull`. Der inferrierte Typ ist der kleinste, in den die Konstante passt, aber mindestens `int`

4.3.5.2. Typkonversionen

Verwendet man Ausdrücke zweier verschiedener Typen in einem Ausdruck, muss einer davon an den anderen angepasst werden. Die Regeln für implizite Typkonversionen in C sind zahlreich. Will man explizit von einem Typen in einen anderen konvertieren, kann man casten:

```c
int x = 5; long y = 7;
y = ((long) x) + y;
x = x + (int) y;
```

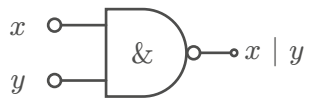
5. ARITHMETISCHE UND LOGISCHE OPERATIONEN

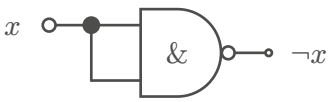
5.1. LOGISCHE OPERATIONEN

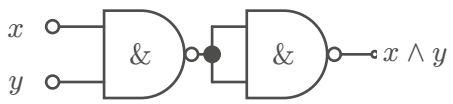
5.1.1. NAND

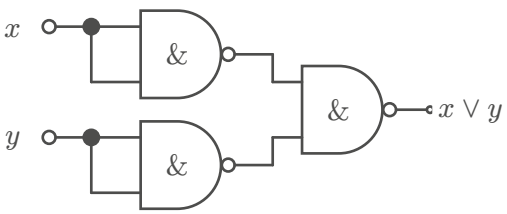
Die Basisoperationen können ausschliesslich aus | (NAND) dargestellt werden. NAND-Bausteine sind technisch leicht als Transistoren zu realisieren.

wzb diese und weitere…









5.1.2. Assembler

Auf Intel 64 ist das Ziel gleichzeitig ein Operand:

$$\begin{aligned}
\texttt{not z} &= z \leftarrow \overline{z} \\
\texttt{and z, q} &= z \leftarrow z \wedge q \\
\texttt{or z, q} &= z \leftarrow z \vee q \\
\texttt{xor z, q} &= z \leftarrow z \oplus q
\end{aligned}$$

Bitweise Operatoren agieren ohne Interpretation der Operanden als Gesamtzahl (signed und unsigned).

5.1.2.1. Grössenangaben

Bei manchen Instruktionen kann die Grösse nicht abgeleitet werden, z.B. `not [myvar]`.

In diesem Fall muss man einen size specifier hinzufügen, z.B. `not word [myvar]`

5.1.3. C

5.1.3.1. Bitwise Operatoren

Die bitweisen Operatoren entsprechen direkt den Assembler-Operationen:

$$\begin{aligned}
\texttt{x = ~ z} &= x \leftarrow \overline{z} \\
\texttt{x = z \& q} &= x \leftarrow z \wedge q \\
\texttt{x = z | q} &= x \leftarrow z \vee q \\
\texttt{x = z ^ q} &= x \leftarrow z \oplus q
\end{aligned}$$

5.1.3.2. Logische Operatoren

Die logischen Operatoren bedeuten etwas anderes als auf Assembler-Ebene:

$$\begin{aligned}
\texttt{x = ! z} &= x \leftarrow \begin{cases} 1 \text{ if } z = 0 \\ 0 \text{ otherwise} \end{cases} \\
\texttt{x = z \&\& q} &= x \leftarrow \begin{cases} 1 \text{ if } z \neq 0 \wedge p \neq 0 \\ 0 \text{ otherwise} \end{cases} \\
\texttt{x = z || q} &= x \leftarrow \begin{cases} 1 \text{ if } z \neq 0 \vee p \neq 0 \\ 0 \text{ otherwise} \end{cases}
\end{aligned}$$

Der Typ dieser Ausdrücke ist immer `int`, der Wert immer `0` oder `1`.

5.2. ARITHMETISCHE OPERATIONEN

5.2.1. Assembler

Aufgrund der Definition des Zweierkomplements sind diese Operationen für signed und unsigned gleich.

$$\begin{aligned}
\texttt{add z, q} &= z \leftarrow z + q \\
\texttt{sub z, q} &= z \leftarrow z - q
\end{aligned}$$

`adc z, q`	$=$	$z \leftarrow z + q + c$	
`sbb z, q`	$=$	$z \leftarrow z - q - c$	
`neg z`	$=$	$z \leftarrow 0 - z$	Zweierkomplement
`inc z`	$=$	$z \leftarrow z + 1$	
`dec z`	$=$	$z \leftarrow z - 1$	

c ist das Carry-Flag, das z.B. bei einer vorhergehenden Addition gesetzt wurde.

5.2.2. C

`z = q + p`	$=$	$z \leftarrow q + p$	
`z = q - p`	$=$	$z \leftarrow q - p$	
`z = - q`	$=$	$z \leftarrow 0 - q$	
`z = ++ q`	$=$	$z \leftarrow q + 1,$	$q \leftarrow q + 1$
`z = q ++`	$=$	$z \leftarrow q,$	$q \leftarrow q + 1$
`z = -- q`	$=$	$z \leftarrow q - 1,$	$q \leftarrow q - 1$
`z = q --`	$=$	$z \leftarrow q,$	$q \leftarrow q - 1$

5.2.3. Links- und Rechts-Shift

Links-Shift um m Bits: $\cdot\, 2^m$
Rechts-Shift um m Bits: $\div\, 2^m$
Im Zweierkomplement mit 4 Bit: $1110b \div 2 = 0111b = 7 \neq -1$. Der Rechts-Shift mit Sign-Extension heisst «Arithmetischer Rechts-Shift».

5.2.3.1. Sign-Extension

Wird eine n-Bit-Zahl auf eine $n + m$-Bit-Speicherstelle kopiert, werden die m oberen Bits auf 0 gesetzt. Bei einer Zahl im Zweierkomplement ändert sich dadurch aber die Bedeutung. Z.B. 4 Bit auf 8 Bit kopieren:

$$b = -1 = 1111b \Rightarrow 0000\ 1111b = 15d \neq -1 = 1111\ 1111b$$

Statt einer 0 muss überall eine 1 kopiert werden. Dieser Vorgang heisst ***sign extension***: das Vorzeichen wird auf alle zusätzlichen Bits ausgedehnt. Die Sign-Extension wird **nicht** in allen Befehlen automatisch ausgeführt (z.B. `mov`).

5.2.3.2. Assembler

Shifts können ähnlich performant wie die elementaren Operationen implementiert werden. Rotates füllen statt mit 0 oder 1 mit den ursprünglichen Bits auf.

`shl z, i`	$=$	$z \leftarrow z \cdot 2^i$	
`shr z, i`	$=$	$z \leftarrow z \cdot 2^{-i}$	z unsigned
`sar z, i`	$=$	$z \leftarrow z \cdot 2^{-i}$	z signed
`rol z, i`	$=$	$z \leftarrow$ Left-Rotate i Bits	
`ror z, i`	$=$	$z \leftarrow$ Right-Rotate i Bits	

5.2.3.3. C

`q << p`	$=$	$z \leftarrow q \cdot 2^p$
`q >> p`	$=$	$z \leftarrow q \div 2^p$

Beim Rechts-Shift wählt der Compiler die Instruktion abhängig vom Typ aus:
– unsigned: `shr`
– signed: `sar`

Rotates können in C nicht direkt ausgedrückt werden.

5.2.4. Multiplikation

5.2.4.1. Unsigned

Unsigned Multiplikation hat nur einen Operanden, der andere ist implizit: `mul z`

Je nach Bit-Grösse von z sind Ergebnis und der andere Operand:

Bits	*Ergebnis MSBs*	*Ergebnis LSBs und zweiter Operand*
64	RDX	RAX
32	EDX	EAX
16	DX	AX
8	AH	AL

5.2.4.2. Signed

`imul` ist das signed Äquivalent zu `mul`. Zusätzlich zu den fixen Registern wie bei `mul` gibt es drei flexiblere Versionen für jede Grösse, bei denen der zweite Operand explizit mitgegeben werden kann: `imul r, z`. Diese Versionen verwenden aber nur die LSBs des Ergebnisses.

5.2.5. Division

Umgekehrt gibt es die Division mit fixen Registern: `div` und `idiv` Der Dividend d verwendet dieselben Register wie das Produkt bei `mul` bzw. `imul`, z.B. `RDX:RAX`.
– Division schreibt den Quotienten $q = {}^{d}/_{z}$ in die LSBs von d, z.B. `RAX`
– Division schreibt den Rest $r = d \bmod z$ in die MSBs von d, z.B. `RDX`

Division in Assembler sollte gemieden werden, da sehr langsame Operation. Sie kann für Zweierpotenzen durch Rechts-Shift und bei anderen Konstanten oft durch Multiplikation oder Umstellen der Formel ersetzt werden.

5.2.6. C

– Multiplikation: `a * b`
– Division: `a / b`
– Rest: `a % b`
– Die richtige Operation (signed oder unsigned) wird aus Typ abgeleitet
– Aber: Typ bleibt gleich, d.h. `int * int` → `int`

5.2.7. Optimierungen

TODO:
W6 S15,16

5.2.7.1. Division

Der GCC ersetzt automatisch die Division durch Zweierpotenzen durch die richtige Shiftoperation:

```
unsigned ux;
ux = ux / 8;
```

wird zu

```
int sx;
sx = sx / 4;
```

wird zu

```
mov eax, [ux]
shr eax, 3
mov [ux], eax
```

```
mov eax, [sx]
sar eax, 2
mov [sx], eax
```

***Beispiel** 3:* **Division durch unsigned konstante**

```
unsigned ux;
ux = ux / 3;
```

wird zu

```
mov eax, [ux]
mov edx, 0xaaaaaaab
mul edx                 ; edx:eax = [ux] * aaaa aaab
mov eax, edx            ; eax = ([ux] * aaaa aaab) / 2^32
shr eax, 1              ; eax = ([ux] * aaaa aaab) / 2^33
mov [ux], eax
```

6. KONTROLLFLUSS

TODO:
W6 S9..12 bit interpretation / encoding comparison

6.1. FLAGS

Flags (Fahnen) sind einzelne Bits, die eine eigenständige Bedeutung haben. Sie liegen in einem gemeinsamen Register `RFLAGS`, das nicht direkt verwendet werden kann. Operationen verändern Flags in unterschiedlichem Masse.

Carry Flag (CF): Gibt Überlauf bei **unsigned** Arithmetik an und entspricht dem Übertrag bei den MSBs

Overflow Flag (OF): Gibt Überlauf bei **signed** Arithmetik an und wird gesetzt, wenn die Vorzeichen nicht mehr stimmen

Zero Flag (ZF): Wird immer gesetzt, wenn das Resultat `0` ist

Sign Flag (SF): Entspricht immer dem höchstwertigen Bit des Resultats

Parity Flag (PF): Wird immer gesetzt, wenn das niederwertigste Byte des Resultats eine gerade Anzahl an gesetzten Bits enthält

Der Prozessor weiss nicht, ob mit signed oder unsigned gearbeitet wird
– Carry Flag und Overflow Flag werden immer beide bestimmt
– Das Programm muss später das richtige Flag verwenden

6.2. CONDITION CODES (CC)

Einige Zustände von Flags oder deren Kombinationen werden als Condition Codes bezeichnet.

Bspw. bedeutet die Bedingung «Above» (kurz: `A`), dass `CF = 0 AND ZF = 0`. Zu jedem Flag gibt es einen CC mit einem Buchstaben:

C $\iff$ CF = 1
Z $\iff$ ZF = 1

Zu vielen CCs gibt es den negierten CC mit vorangestelltem N

NC $\iff$ CF = 0
NA $\iff$ CF = 1 OR ZF = 1

Condition Codes werden in bestimmten Befehlen dazu verwendet, eine Bedingung anzugeben. Der Befehl wird nur dann ausgeführt, wenn in diesem Moment die Flags genau so gesetzt sind, wie der Condition Code angibt.

Beispiel 4: **Conditional Move**

```
dec rax                 ; sets ZF = 1 if rax == 0
cmovz rax, [q]          ; sets rax = [q] if rax == 0
```

6.2.1. Unsigned

Die Differenz $a - b$ zeigt an, ob a kleiner, grösser oder gleich b ist:

Fall	*Flags*	*Begründung*
$a = b$	$\mathrm{ZF} = 1$	$a - b = 0$
$a < b$	$\mathrm{CF} = 1$	$a - b$ ergibt einen (unsigned) Überlauf
$a \geq b$	$\mathrm{CF} = 0$	$a \geq b \Leftrightarrow \overline{a < b}$
$a \leq b$	$\mathrm{ZF} = 1 \vee \mathrm{CF} = 1$	$a \leq b \Leftrightarrow a < b \vee a = b$
$a > b$	$\mathrm{ZF} = 0 \wedge \mathrm{CF} = 0$	$a > b \Leftrightarrow \overline{a \leq b}$

6.2.2. Signed

Fall	*Flags*	*Begründung*
$a = b$	$\mathrm{ZF} = 1$	$a - b = 0$
$a < b$	$\mathrm{OF} \neq \mathrm{SF}$	$(a < 0 \wedge b > 0) \Leftrightarrow \mathrm{OF} = 1 \wedge \mathrm{SF} = 0$ Overflow $a < b \Leftrightarrow \mathrm{OF} = 0 \wedge \mathrm{SF} = 1$ No overflow
$a \geq b$	$\mathrm{OF} = \mathrm{SF}$	$a \geq b \Leftrightarrow \overline{a < b}$
$a \leq b$	$\mathrm{ZF} = 1 \vee \mathrm{OF} \neq \mathrm{SF}$	$a \leq b \Leftrightarrow a < b \vee a = b$
$a > b$	$\mathrm{ZF} = 0 \wedge \mathrm{OF} = \mathrm{SF}$	$a > b \Leftrightarrow \overline{a \leq b}$

6.2.3. Intel CCs

	$a < b$			$a > b$		
signed	Less	L	$\mathrm{SF} \neq \mathrm{OF}$	Greater	G	$\mathrm{ZF} = 0 \wedge \mathrm{OF} = \mathrm{SF}$
unsigned	Below	B	$\mathrm{CF} = 1$	Above	A	$\mathrm{ZF} = 0 \wedge \mathrm{OF} = 0$

CC	*Name*	*Flags*	*CC*	*Name*	*Flags*
A	Above	$\mathrm{CF} = 0 \wedge \mathrm{ZF} = 0$	AE	Above or Equal	$\mathrm{CF} = 0$
B	Below	$\mathrm{CF} = 1$	BE	Below or Equal	$\mathrm{CF} = 1 \vee \mathrm{ZF} = 1$

G	Greater	$\mathrm{ZF} = 0 \wedge \mathrm{SF} = \mathrm{OF}$	GE	Greater or Equal	$\mathrm{SF} = \mathrm{OF}$
L	Less	$\mathrm{SF} \neq \mathrm{OF}$	LE	Less or Equal	$\mathrm{ZF} = 1 \vee \mathrm{SF} \neq \mathrm{OF}$
PE	Parity Even	$\mathrm{PF} = 1$	PO	Parity Odd	$\mathrm{PF} = 0$
E	Equal	$\mathrm{ZF} = 1$			

6.3. JMP

Der Befehl `jmp d` setzt $\mathrm{RIP} \leftarrow \mathrm{RIP} + d$ bezogen auf RIP nach dem Befehl selbst. d ist eine 8- oder 32-Bit-Zahl, die auf 64-Bit sign-extended wird. Das Programm springt um d: vorwärts, wenn d positiv, oder rückwärts wenn d negativ ist.

Verwendet man ein Label, berechnet der Assembler die korrekte relative Adresse.

6.4. VERGLEICHE

6.4.1. cmp

Die Operation `cmp` entspricht der Operation `sub` ohne Veränderung der Operanden.

6.4.2. test

Die Operation `test` entspricht der Operation `and` ohne Veränderung des Ziel-Operanden, setzt aber trotzdem die Flags.

6.5. BEDINGTE ANWEISUNGEN

Es gibt auf Intel-Prozessoren 3 Familien von bedingten Operationen
– `cmovcc`: Conditional MOV
– `jcc`: Conditional JMP
– `setcc`: Schreibt 1 in das 8-Bit grosse Ziel, wenn CC erfüllt, sonst 0

6.5.1. Bedingte Sprünge

Bedingte Sprünge benötigen einen Condition Code und springen nur dann, wenn der CC erfüllt ist, ansonsten fährt der Prozessor mit der nächsten Instruktion fort.

6.5.1.1. C

Verzweigungen werden in C mit `if` implementiert. Der Body wird genau dann ausgeführt, wenn die Bedingung nicht `0` ist.

Falls ein `else` vorhanden ist, kommt am Ende des ersten Bodys ein unbedingter Sprung.

***Beispiel** 5:*

```c
if (ux < 3) {
  // if-body ...
} else {
  // else-body ...
}
```

```asm
  mov eax, [ux]
  cmp eax, 2
  ja else_body
  ; if-body
  jmp after_if

else_body:
```

```
  ; else-body
after_if:
```

6.5.2. Schleifen

Springen wir rückwärts, können wir eine Schleife implementieren:

```
loop:
  ; body
  dec rcx
  jnz loop
```

6.5.2.1. C

In C ist die do-while-Schleife das Analogon zur Schleife in Assembler:

```
do {
  /* body */
} while (condition);
```

Bei der while-Schleife wird vor jedem Ausführen die Bedingung geprüft, somit ein mal öfter als der Body.

***Beispiel** 6:*

```
int i = 23;
while (--i) {
  /* body */
}
```

```
  mov rcx, 23
  jmp condition
loop:
  ; body
condition:
  dec rcx
  jnz loop
```

6.6. POINTER-ADDITION

Die Addition eines Integers zu einem `T*`-Pointer berücksichtigt die Grösse `sizeof(T)` (genauso bei `++`). Z.B. Bei `int *p = (int *)0x20;` ist `p+1 == 0x24` und `p+2 == 0x28`, usw.

6.7. FUNKTIONEN

In Assembly sind funktionen nichts anderes als labels mit jumps, genauso wie bei loops.

6.7.1. Parameter

Parameter können auf verschiedene Arten angegeben werden.

Art	*Schwierigkeiten*	*Beispiel*

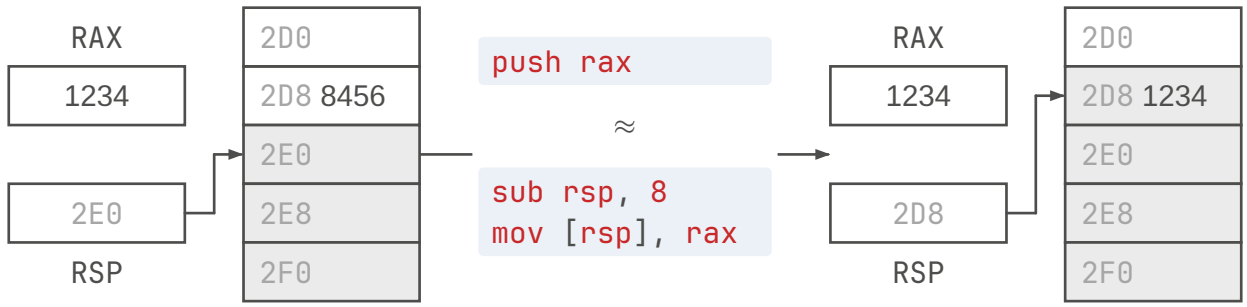
Register	– Begrenzte Anzahl Register – Schwierig, Wenn aus einer Sequenz eine andere Sequenz aufgerufen wird	``` ; ... mov rbx, 5 jmp myfun return_to: myfun: mov rax, rbx ; do some stuff ```
Globale Variablen	– Für jede Sequenz muss Speicher für die Parameter statisch reserviert werden – Für jede Sequenz gibt es jeden Parameter genau einmal – Rekursion würde Parameter überschreiben – Paralellisierungsprobleme	``` section .data op db 5 section .text global _start _start: jmp myfun myfun: mov rax, [op] ; do some stuff ```
Stack	– Falls `call` verwendet wird, wird die Rücksprungadresse auch auf den Stack gelegt	``` ; ... push 5 jmp myfun myfun: pop rax ; do some stuff ```

Damit der Prozessor weiss, wohin nach Ausführung der Funktion zurückzuspringen ist, muss die Rücksprungadresse irgendwo hinterlegt werden. Dies kann auch jeweils mit Register, globalen Variablen oder dem Stack geregelt werden.

6.7.2. Stack

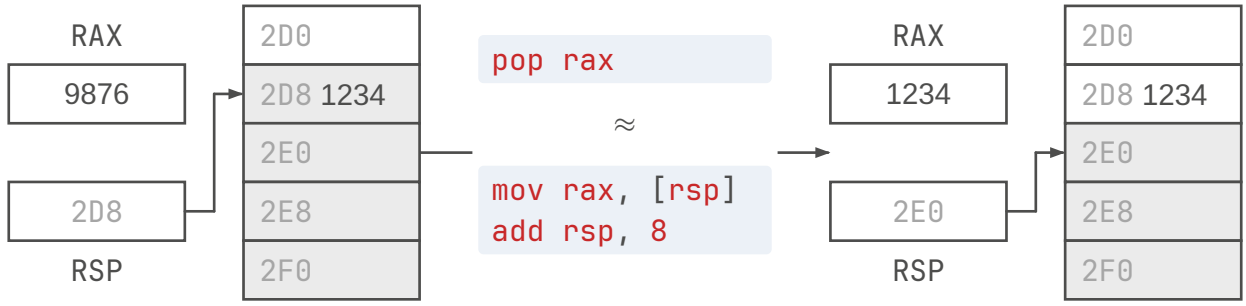
Das Konzept «Stack» umfasst einen Stackpointer und Operationen `push` und `pop`: Der Stackpointer (`rsp`) zeigt immer auf das zuletzt abgelegte Element.

`push q` kopiert das Element aus `q` oben auf den Stack und passt Stackpointer an.



`pop z` kopiert das oberste Element vom Stack nach `z` und passt den Stackpointer an.

`push rax` entspricht nicht ganz `sub`: die Flags werden nicht gesetzt.



Elemente auf dem Stack sind jeweils 8 Byte gross. Das «oberste» Element liegt an der **niedrigsten** Adresse, d.h. der Stack wächst in Richtung der niedrigen Adressen. Der Prozessor überprüft keine obere oder untere Grenze.

`pop rax` entspricht nicht ganz `add`: die Flags werden nicht gesetzt.

6.7.2.1. Nutzen

Der Stack bietet einen Mechanismus, um automatisch Speicher zu allozieren. Auf dem Stack kann man z.B. Zwischenwerte, Argumente, Rückgabewerte oder die Rücksprungadresse speichern.

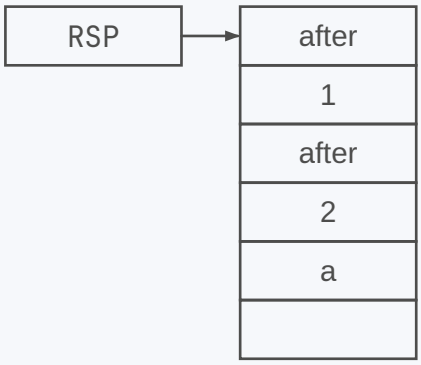
***Beispiel** 7:*

```
  ; n in rax, S(n) in rax
S:
  test rax, rax        ; n == 0?
  jz final             ; if (n == 0) jump back
  push rax             ; push n
  dec rax              ; rax ← n-1
  push after           ; push retaddr for S(n-1)
  jmp S                ; rax ← S(n-1)

after:
  pop rcx              ; rcx ← n
  add rax, rcx         ; rax ← S(n-1) + n

final:
  pop rsi              ; rsi ← retaddr
  jmp rsi              ; jump back
```

Stack am Beispiel für $n = 2$, IP an Label S bei $n = 0$:



6.7.3. CALL/RET

Vorteile: Rücksprungadresse ist automatisch bekannt, nämlich ist es die direkt nach dem Sprung

call a:

Nachfolgende Adresse r auf den Stack und Sprung an a

`call S` =
```
push cont
mov rip, S
cont:
```

ret: Adresse r vom Stack und Sprung an r

6.7.4. Funktion

Der Bereich vom Einsprungslabel `f` bis zum letzten `RET` heisst «Funktion `f`» In der Praxis gibt es oft mehr als ein `RET` pro Funktion.

Aufruf der Funktion: Verwendung von `CALL`

Rücksprung aus der Funktion: Ausführung von `RET`

Caller: Aufrufende Funktion

Callee: Aufgerufene Funktion

6.7.5. Argumente

Argumente werden nacheinander auf den Stack gepusht, Rücksprungadresse kommt **zuletzt** auf den Stack durch `CALL`. Argumente liegen unter der Rücksprungadresse und können nicht gepoppt werden. Stattdessen bleiben Argumente bis zum `RET` auf dem Stack.

6.7.6. Frame Pointer

Durch konsekutive `CALL` Operationen springt der Stackpointer ständig hin und her. Um die Adressen der Argumente nicht relativ zum Stackpointer verwalten zu müssen gibt es einen stabilen Pointer: den Frame Pointer. Auf Intel 64: `RBP` (Base Pointer)

Am Beginn der Funktion steht der ***Prolog***:

```
push rbp          ; push caller's frame pointer
mov rbp, rsp      ; rbp ← rsp
```

Am Ende der Funktion steht der ***Epilog***:

```
mov rsp, rbp      ; rsp ← rbp, only if rsp changed
pop rbp           ; restore caller's frame pointer
```

***Beispiel** 8:*

```
f:
  push [x]
  call g
  ; ...
g:
  push rbp
  mov rbp, rsp
  mov rcx, [rbp + 0x10]   ; access argument on stack
  mov rsp, rbp
  pop rbp
  ret
```

6.7.7. Calling Convention

Vereinbarung zwischen Caller und Callee darüber:
– wo Argumente übergeben werden
– wo Rückgabewerte übergeben werden
– welche Register die Funktion verändern darf

– wer den Stackframe für Argumente wieder abbaut
– wer den Stackframe für lokale Variablen ab- und aufbaut

Sie
– wird durch Betriebssystem (zwischen Programmen) bzw. Compiler (innerhalb eines Programms) bestimmt
– ist für verschiedene Betriebssysteme oft verschieden
– ist für verschiedene Compiler oft verschieden
– ist ein Hauptgrund, warum verschiedene Compiler/Betriebssysteme oft nicht kompatibel sind

Die Calling Convention bestimmt die Performance des Gesamtsystems als Tradeoff:
– Werden mehr Register verwendet, können weniger Register für Rechnungen verwendet werden
– Wird mehr Stack verwendet, werden mehr langsame Speicherzugriffe generiert

Es gibt also kein allgemeines Optimum

6.7.8. Zwischenergebnisse

Der Stack kann auch für Zwischenergebnisse verwendet werden. Anstatt jedes Zwischenergebnis einzeln zu pushen, kann man den benötigten Speicher mit einer Instruktion (de)allozieren:

```
sub rsp, 0x20          ; allocates 0x20 bytes on stack
add rsp, 0x20          ; deallocates 0x20 bytes on stack
```

Die Zwischenergebnisse liegen dann **über** derm Framepointer

```
mov rcx, [rbp - 0x8]  ; access memory for results
```

6.7.9. C

Aller ausführbarer Code muss in Funktionen stehen. Der Rückgabewert wird **immer kopiert**: Je nach Grösse und Calling-Convention in Registern oder auf dem Stack.

Eine Funktion ohne Parameterliste kann mit Argumenten aufgerufen werden. Eine Funktion mit Parameterliste `void` kann nur mit leerer Argumentenliste aufgerufen werden.

```
void f ();
void g (void);

f();              // OK
g();              // OK
f(1, 2, 3);       // OK
g(1, 2, 3);       // ERR
```

Parameter werden je nach Calling Convention des Compilers in Registern oder auf dem Stack übergeben. Argumente werden auch immer kopiert.

6.7.9.1. Lifetimes

Jedes Objekt in C hat eine Lebensdauer (lifetime), während derer Speicherplatz mit dem richtigen Wert garantiert ist:

Globale Variablen: so lange, wie das Programm läuft

Argumente: bis zum Ende der Ausführung der Funktion

Rückgabewerte: vom Ende der Ausführung der Funktion bis zur Auswertung durch den Aufrufer

Jede andere Variable: vom Zeitpunkt ihrer Definition bis zum Ende des Blocks, in dem sie definiert wurde

6.7.9.2. Variablen

Werden bei jedem Aufruf auf dem Stack oder in Registern neu angelegt. Werden sie nicht initialisiert, ist ihr Wert, anders als bei globalen Variablen, nicht definiert.

Globale Variablen haben eine fixe Adresse im Speicher.

Rückgabe lokaler Variablen ist zwar erlaubt aber führt zu nicht definiertem Verhalten.
– Code, der diese Adressen schreibt, korrumpiert den Stack
– Code, der von diesen Adressen liest, liest ganz andere Daten

```c
int * f (int **x) {
  int y = 123;
  *x = &y;
  return &y;
}
```

6.7.9.3. Funktionspointer

Jede Funktion hat eine Adresse, die auch in einer Variablen gespeichert werden kann.

```c
int f (int x, int y);
int g (int x, int y);
int (*p) (int, int) = &f;
p = g;
int i = (*p) (1, 2);        // Ruft Funktion auf, deren Adresse in p ist
int j = p (3, 4);           // Alternativer Aufruf
```

6.7.9.4. printf

Jede Konvertierungs-Spezifikation beginnt mit % gefolgt von einem Format-Zeichen:

Begriff	*Definition*
%i	Die nächsten `sizeof(int)` Bytes vom Stack als signed Dezimalzahl
%u	Die nächsten `sizeof(int)` Bytes vom Stack als unsigned Dezimalzahl
%x, %X	Die nächsten `sizeof(int)` Bytes vom Stack als Hexadezimalzahl
%li	Die nächsten `sizeof(long)` Bytes vom Stack als signed Dezimalzahl
%lli	Die nächsten `sizeof(long long)` Bytes vom Stack als signed Dezimalzahl
%p	Die nächsten `sizeof(void *)` Bytes vom Stack als Pointer (hexadezimal)
%s	Interpretiert die nächsten `sizeof(char *)` Bytes als Pointer auf einen null-terminierten String